



Master's Degree Program

Field of Study: Advanced Analysis – Big Data

Thi Van Trang Le

Number ID : 140036

Process Intelligence with Python Language

Master's Thesis

Under the supervision of

dr hab. Michała Ramsza

Institute of Mathematical Economics

Warszawa 2026

Contents

1	Introduction	4
1.1	Background and motivation	4
1.2	Research objectives and research question	5
1.3	Relevance and academic context	6
1.4	Methodology	7
1.5	Structure of the thesis	8
2	The tools available in Python	9
2.1	Core process mining libraries	9
2.2	Predictive and AI-oriented tools	9
2.3	Simulation and optimization tools	10
2.4	Visualization libraries	10
2.5	Complementary data processing libraries	10
2.6	Summary	11
3	Worked examples	12
3.1	Introductory synthetic example	12
3.2	BPI Challenge 2019: the case study	13
3.2.1	Data source, format, and structure	13
3.2.2	Purpose and analytical workflow	13
3.2.3	Implementation and results	13
3.2.4	Discussion of results	17
3.3	Advanced example: loops, variants, and performance analysis	18
3.3.1	Variant analysis	18
3.3.2	Loop and rework detection	18
3.3.3	Throughput time and performance KPIs	18
3.3.4	Implementation	19
3.3.5	Example 2 advanced (OCEL): Loops + Performance (Time)	19
4	Conclusions	26
4.1	Summary of findings	26
4.2	Limitations	26
4.3	Directions for future research	27
	Bibliography	28
	List of figures	29
	List of tables	30

Chapter 1

Introduction

1.1 Background and motivation

Modern organizations carry out much of their work through digital platforms such as Enterprise Resource Planning (ERP) systems, Customer Relationship Management (CRM) tools, and dedicated workflow management applications. As these systems operate, they automatically generate a continuous stream of structured records that capture every action performed within a business process. Each record typically stores information about which activity was performed, who performed it, when it occurred, and which business object it concerned. Over time, these records accumulate into large repositories of operational data that reflect the actual behavior of business processes in precise detail.

This type of data offers a qualitatively different view of business operations compared with traditional management reports. A management report typically presents aggregated statistics — average throughput times, error rates, or resource utilization figures — computed from a simplified model of how the process should work. By contrast, the raw event data generated by a digital information system describes how the process actually works, case by case and activity by activity. The gap between these two views is often significant. Studies have consistently shown that real business processes deviate from their intended designs in ways that are difficult to detect from aggregate data alone (Aalst, 2022). Bottlenecks, rework cycles, policy violations, and unofficial workarounds are all examples of behavior that is invisible in summary reports but clearly visible in event data when it is analyzed correctly.

Process mining is the scientific discipline that bridges this gap. It applies computational methods to event logs — collections of recorded process events — in order to discover how a process is structured, verify whether it follows established rules, and measure its operational performance. Since its formal introduction in the late 1990s, process mining has grown from a niche research topic into a well-established field with a substantial body of theory, a dedicated international conference, and a growing number of commercial and open-source software tools (Aalst, 2022). Today it is applied in industries including healthcare, finance, logistics, and manufacturing, where it supports tasks such as compliance auditing, operational improvement, resource management, and process redesign.

Process intelligence is a broader term that encompasses process mining but extends it in several important directions. While process mining is primarily retrospective — it analyzes data that has already been recorded — process intelligence also includes real-time monitoring of running processes, predictive modeling to anticipate future outcomes, simulation of alternative process designs, and optimization of process parameters. This combination of retrospective analysis, real-time awareness, and forward-looking

capabilities represents the current state of the art in data-driven process management. Organizations that adopt process intelligence are not limited to understanding what went wrong in the past; they can also intervene in ongoing processes and proactively improve future performance.

Python has become the dominant language for data science and analytical computing over the past decade, and this has made it a natural platform for implementing process mining and process intelligence workflows. The Python ecosystem includes not only general-purpose tools for data manipulation, statistical analysis, and machine learning, but also a growing set of libraries specifically designed for process mining. The most prominent of these is PM4Py, an open-source library that implements the full process mining workflow — from loading and exploring event logs to discovering process models, checking conformance, and analyzing performance. Other libraries extend these capabilities to include simulation, prediction, and interactive visualization. Together they form an ecosystem that makes it possible to conduct end-to-end process intelligence analysis within a single Python environment, without reliance on proprietary platforms or specialized software.

Despite this growth, the Python process mining ecosystem remains fragmented and difficult to navigate for a practitioner who is encountering it for the first time. Libraries have overlapping functionalities, varying levels of documentation, and different assumptions about data formats and analytical workflows. No systematic and practical comparison of these tools currently exists that covers the breadth of available libraries and illustrates their use with concrete, reproducible examples. There is therefore a clear need for a structured review that organizes these tools, explains their theoretical foundations, and demonstrates their capabilities with working implementations.

1.2 Research objectives and research question

The central research question of this thesis is:

Which Python tools are available for process mining and process intelligence, and how suitable are they for key analytical tasks such as process discovery, conformance checking, and performance analysis?

To address this question, the thesis pursues four specific objectives:

1. To introduce and formalize the mathematical concepts that underlie process mining — including event logs, traces, directly-follows relations, process models, and conformance measures — so that the capabilities and limitations of different tools can be understood and compared on a rigorous basis.
2. To survey the main Python libraries available for process mining and process intelligence, organized by their primary function, and to describe their features, strengths, and limitations in a structured way.
3. To demonstrate the use of selected tools through practical worked examples, including a synthetic introductory example and a real-world case study based on a publicly available benchmark dataset.

4. To draw conclusions about the current state of the Python process mining ecosystem and to identify gaps and directions for future development.

The scope of the thesis is limited to Python tools and to the three core process mining tasks — discovery, conformance checking, and performance analysis — along with adjacent capabilities such as simulation and predictive monitoring. The focus is on open-source tools that are accessible to practitioners and researchers without requiring commercial licenses, with the exception of PyCelonis, which is included because it represents an important category of commercial process intelligence platform with a Python interface.

1.3 Relevance and academic context

Process mining as a scientific field was formally established around the work of Wil van der Aalst and colleagues at Eindhoven University of Technology in the late 1990s and early 2000s. The field has since grown into a mature discipline with dedicated academic infrastructure: the International Conference on Process Mining (ICPM) is held annually, and the IEEE Task Force on Process Mining has produced a formal manifesto defining the scope and principles of the field. The textbook by van der Aalst (Aalst, 2022) provides a comprehensive reference for both the theoretical foundations and the practical applications of process mining and is the primary academic reference for this thesis.

The BPI Challenge, organized annually in connection with ICPM, provides publicly available benchmark datasets from real organizations and invites researchers to analyze them using process mining methods. These datasets have become standard benchmarks in the process mining literature and are widely used to compare algorithms, tools, and analytical approaches. The BPI Challenge 2019 dataset, which is used as the main case study in this thesis, was collected from a real procurement process at a large multinational company. It contains nearly 1.6 million events and represents one of the most complex datasets in the BPI Challenge series. It has been analyzed in numerous academic papers, which makes it a suitable choice for a thesis that aims to demonstrate process mining techniques in a realistic setting.

The Python ecosystem for process mining has developed rapidly since the initial release of PM4Py in 2018. Earlier process mining software was implemented primarily in ProM, a Java-based platform that remains widely used in academic research but requires specialized knowledge to extend and deploy. The availability of a Python implementation of the core algorithms significantly lowered the barrier to entry and enabled process mining to be integrated with the broader Python data science ecosystem, including libraries for machine learning, visualization, and statistical analysis. Since then, additional libraries have been released to support specific tasks such as streaming process mining, simulation-based optimization, and predictive monitoring. However, the ecosystem remains less mature and less comprehensively documented than the equivalent Java-based ProM ecosystem, which makes a structured review particularly timely.

This thesis contributes to this context by providing a structured, practitioner-oriented review of the Python process mining ecosystem, with particular attention to PM4Py and its extensions. The practical examples are implemented as Jupyter notebooks, which allows all steps to be reproduced and verified by the reader. This reproducibility is an important feature of the work, as it ensures that the results reported in Chapter 4 can be independently validated and extended by other researchers or practitioners.

1.4 Methodology

This thesis is a structured review combined with empirical demonstration. The work follows four sequential phases.

Phase 1: Theoretical foundation. The mathematical concepts required to understand process mining are introduced and formalized in Chapter 2. The primary source for these definitions is van der Aalst (Aalst, 2022). Establishing this theoretical foundation before describing tools is important because it provides a common vocabulary for comparing different implementations and for interpreting the results of the practical examples.

Phase 2: Tool survey. Python libraries are identified through a combination of academic sources — papers and conference proceedings from ICPM and related venues — the Python Package Index (PyPI), GitHub repositories related to process mining and process intelligence, and cross-references within the PM4Py documentation. For each tool, the following aspects are examined: the primary analytical task it supports, the data formats it accepts, the algorithms it implements, and the level of active maintenance and community support as indicated by recent commit history and documentation quality. Tools are organized into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages.

Phase 3: Practical implementation. Two examples are implemented in Python using PM4Py as the primary tool, with supporting libraries as needed. The first is a synthetic introductory example that constructs a small event log by hand and illustrates the basic process mining workflow on data where the correct answer is known in advance. The second is a case study using the BPI Challenge 2019 dataset, which applies the same workflow to a large, complex, real-world event log and reports the resulting process model and performance statistics. Both examples are implemented as Jupyter notebooks, which are embedded directly in the thesis so that the code, outputs, and commentary are presented together in a transparent and reproducible format.

Phase 4: Synthesis and conclusions. The results of the tool survey and the practical examples are combined in Chapter 5, where the research question is answered, the limitations of the work are acknowledged, and directions for future research are proposed.

The methodology is descriptive and demonstrative rather than evaluative in a strict experimental sense. No formal benchmarking is performed — the thesis does not measure and compare the runtime, fitness scores, or precision values of multiple discovery

algorithms on the same dataset. Such a comparison would require a controlled experimental design and falls outside the scope of this work. The contribution instead lies in the structured organization, explanation, and demonstration of the available tools.

1.5 Structure of the thesis

The remainder of this thesis is organized as follows.

Chapter 2 introduces the mathematical foundations of process mining. It formally defines events, traces, and event logs, explains directly-follows relations and their role in process discovery, describes the main process model representations used in this thesis — the directly-follows graph, the process tree, and the Petri net — and introduces the concepts of conformance checking and performance analysis. It also explains the structure of the Object-Centric Event Log (OCEL) format and provides an exploratory description of the BPI Challenge 2019 dataset.

Chapter 3 reviews the main Python tools available for process mining and process intelligence. The tools are organized into five categories — core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages — and each category is described in terms of the tasks it supports and the tools it contains. The chapter concludes with a comparative summary table.

Chapter 4 presents three worked examples. The first is a synthetic introductory example that demonstrates the basic process mining workflow on a small hand-constructed dataset. The second is a full case study based on the BPI Challenge 2019 dataset, covering data loading, exploratory analysis, flattening, process discovery, and result interpretation. The third extends the case study to include variant analysis, loop detection, and throughput time distribution analysis.

Chapter 5 summarizes the main findings of the thesis, discusses its limitations, and proposes directions for future research.

Chapter 2

The tools available in Python

The Python ecosystem for process mining and process intelligence has grown considerably over the past decade. It now covers a wide range of analytical tasks, from loading and exploring event logs to discovering process models, checking conformance, predicting future outcomes, and simulating alternative process designs. This chapter provides a structured overview of the main Python tools available for these purposes. The tools are organized into five groups: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages.

2.1 Core process mining libraries

The most important library in the Python process mining ecosystem is PM4Py. It is an open-source package developed by the Process Intelligence Solutions group and supported by Fraunhofer FIT. PM4Py covers the complete process mining workflow: it can read and write event logs in multiple formats including XES and OCEL, compute statistics such as variant frequencies and directly-follows graphs, discover process models using algorithms such as the Alpha algorithm and the Inductive Miner, perform conformance checking using both token-based replay and alignment-based methods, and analyze process performance using timestamp data. It also provides dedicated support for object-centric event logs, which makes it particularly suitable for complex processes where multiple business entities are involved. PM4Py is the primary tool used in the practical examples of this thesis.

A streaming extension of PM4Py, PM4Py-Streaming, is available for settings where event data arrives continuously rather than as a complete stored log. It enables online process discovery and monitoring, which is relevant for real-time process intelligence applications.

Two further libraries are worth noting in this category. Simod is a tool for simulation-based process optimization that takes BPMN models as input and generates optimized simulation parameters. It is particularly useful when the goal is not only to understand the current process but also to test alternative designs. PMLab is a research-oriented toolkit that supports a range of process discovery and conformance algorithms, mainly aimed at academic experimentation rather than production use.

2.2 Predictive and AI-oriented tools

Several Python tools connect process mining with machine learning and predictive analytics. ML4ProM is a framework that combines process mining techniques with machine learning models to support predictive process monitoring. It can be used to predict the remaining time of a running case, the next activity, or the likelihood of a process deviation. A related package, pm4pyml, extends PM4Py with scikit-learn

compatible pipelines, allowing machine learning models to be trained directly on event-log features produced by PM4Py.

PyCelonis is the Python API for the Celonis commercial process intelligence platform. It provides access to Celonis functionality including process analysis, action flows, and RPA integration. While Celonis is a proprietary tool, PyCelonis allows Python-based automation and integration within the Celonis environment, which makes it relevant for practitioners working in organizations that use this platform.

The bpmn-python library and related tools support the extraction and manipulation of BPMN process models, enabling integration between process mining outputs and workflow automation systems.

2.3 Simulation and optimization tools

SimPy is a widely used Python library for discrete-event simulation. Although it is not specific to process mining, it can be used to model and simulate business process behavior based on parameters estimated from event logs. This makes it a useful complement to process mining tools when the goal is to test hypothetical process changes before implementing them.

BPSim is a simulation standard compatible with BPMN models, and it can be used together with Simod to run structured process simulations. ProcessOptimizer, built on Bayesian optimization techniques from scikit-optimize, provides a framework for tuning process parameters in order to improve performance outcomes. These tools extend the scope of analysis from retrospective understanding toward forward-looking optimization, which is a key characteristic of process intelligence.

2.4 Visualization libraries

Visualization is an important part of process analysis, both for exploring event data and for communicating results. PM4Py includes built-in visualization capabilities for directly-follows graphs, Petri nets, process trees, and performance-annotated models, using Graphviz as its rendering backend. Graphviz itself is a widely used open-source tool for drawing directed graphs and is used by several process mining libraries for model rendering.

For more interactive and flexible visualization, Plotly and Dash provide a framework for building web-based dashboards that can display process performance metrics, variant distributions, and time-based statistics. NetworkX supports graph-based analysis and visualization, which is useful for examining the structural properties of directly-follows graphs and process networks. These libraries are typically used alongside PM4Py rather than as replacements for it.

2.5 Complementary data processing libraries

A number of general-purpose Python libraries are essential for working with process mining tools in practice. The most important is pandas, which provides the tabular data structures used to store and manipulate event data before and after it is processed by

PM4Py. Most event logs are initially loaded as pandas DataFrames and then converted into PM4Py event-log objects for analysis.

NumPy supports numerical computation and is used internally by most process mining and machine learning libraries. Scikit-learn provides a wide range of machine learning algorithms and is the standard tool for building predictive models on top of features extracted from event logs. For more complex prediction tasks, gradient boosting libraries such as XGBoost and LightGBM offer strong performance on tabular process data. Deep learning frameworks such as TensorFlow and PyTorch are used in more advanced research applications, for example when training neural networks on sequences of process events for next-activity prediction.

2.6 Summary

The table below provides a concise overview of the tools discussed in this chapter, organized by category.

Category	Tool	Primary use
Core process mining	PM4Py	Discovery, conformance, performance, OCEL
	PM4Py-Streaming	Real-time online process mining
	Simod	Simulation-based process optimization
	PMLab	Research-oriented discovery and conformance
Predictive / AI	ML4ProM	Predictive process monitoring
	pm4pymml	ML pipelines on PM4Py features
	PyCelonis	Integration with Celonis platform
Simulation	SimPy	Discrete-event simulation
	ProcessOptimizer	Bayesian parameter optimization
Visualization	Graphviz	Graph and model rendering
	Plotly / Dash	Interactive dashboards
	NetworkX	Graph analysis and visualization
Data processing	pandas	Event log manipulation
	scikit-learn	Machine learning on process features
	XGBoost / LightGBM	Gradient boosting for prediction

Among these tools, PM4Py stands out as the most comprehensive and actively maintained library for process mining in Python. It is the tool used throughout the practical examples in this thesis. The other libraries are either specialized extensions of PM4Py, tools for adjacent tasks such as simulation and prediction, or general-purpose packages that support the data processing workflow. Together, they form a sufficiently complete ecosystem for conducting end-to-end process intelligence analysis within a single Python environment.

Chapter 3

Worked examples

3.1 Introductory synthetic example

This section presents a simple synthetic example to illustrate the basic structure of an event log and demonstrate how process mining works in practice. Instead of using real organizational data, a small artificial dataset is constructed to show how process events can be represented in a table and then analyzed.

The synthetic dataset represents a simplified purchasing process. Each row in the table corresponds to a single event, and the dataset contains three core attributes: the case identifier, the activity name, and the timestamp. These are the minimum attributes required to perform process mining.

Case ID	Activity	Timestamp
1	Create order	2024-01-01 09:00
1	Approve order	2024-01-01 10:00
1	Send invoice	2024-01-01 11:00
2	Create order	2024-01-02 09:15
2	Approve order	2024-01-02 10:30
2	Send invoice	2024-01-02 11:20
3	Create order	2024-01-03 08:45
3	Approve order	2024-01-03 09:50
3	Send invoice	2024-01-03 10:40

The table contains three process instances, each following the same sequence of activities: Create order, Approve order, and Send invoice. In process mining terminology, this sequence is called a trace. Since all three cases share the same trace, the event log contains only one distinct variant. The directly-follows graph for this log would show two relations: Create order $\rightarrow$ Approve order and Approve order $\rightarrow$ Send invoice, both with a frequency of three.

This example is intentionally simple. Its purpose is to show how a plain table becomes the input for process mining procedures in Python. Using PM4Py, a pandas DataFrame with these three columns can be converted into an event-log object, from which a directly-follows graph and a Petri net can be discovered in just a few lines of code. The resulting Petri net for this log is a simple sequence of three transitions connected by places, which correctly reflects the linear structure of the purchasing process shown in the table.

3.2 BPI Challenge 2019: the case study

3.2.1 Data source, format, and structure

The BPI Challenge 2019 dataset is a publicly available event log released as part of the annual BPI Challenge competition, which is organized in connection with the International Conference on Process Mining. The data comes from a real procurement process at a large multinational company and has been widely used in academic research on process mining. The dataset is provided in Object-Centric Event Log (OCEL) format, which makes it particularly well suited for this thesis because it demonstrates a more advanced data representation than the classical single-case format.

The dataset contains approximately 1,595,923 events and 330,685 objects distributed across four object types: purchase orders (PO), purchase order items (POItem), resources (Resource), and vendors (Vendor). Each event is associated with one or more objects from these types and carries a timestamp and an activity name. Additional business context attributes are also present at the event level, including company code, document type, item category, spending area, and vendor details. These attributes make the dataset rich enough for both control-flow analysis and performance or organizational analysis.

In Python, the dataset is loaded using the `pm4py.read_ocel` function, which parses the OCEL file and returns an object-centric event-log structure. This structure preserves the full relationships between events and multiple object types, as described in Chapter 2.

3.2.2 Purpose and analytical workflow

The purpose of this case study is to demonstrate how PM4Py can be applied to a real, large-scale event log to extract a meaningful process model and derive practical insights. The analysis follows a standard process mining workflow and covers the key steps introduced in Chapter 2: data loading, exploratory analysis, flattening to a case-centric format, process discovery, and result interpretation.

A key challenge in this case study is the scale and complexity of the data. With nearly 1.6 million events and multiple object types, the dataset cannot be analyzed as a simple trace collection without some preprocessing. The workflow therefore begins with basic exploratory statistics, continues with a flattening step that selects the most frequent object type as the case identifier, and then applies sampling to reduce the log to a computationally manageable size before running the discovery algorithm. This sequence of steps is representative of how process mining is applied in practice to real organizational data.

3.2.3 Implementation and results

The following notebook implements the full workflow for this case study. It covers: loading the OCEL file, computing basic statistics on events and objects, identifying the most common object type, flattening the log to a classical event-log format, sampling

the log, discovering a directly-follows graph and a Petri net using the Inductive Miner, and saving the resulting visualizations.

First, we load all required modules and provide specific configuration for the example.

```
In [2]: import os
import shutil
import warnings
import pm4py
```

Configuration cell.

```
In [ ]: # Data file
OCEL_PATH = r"..\..\data\BPIC19.jsonocel"

# Speed-up for big dataset
USE_SAMPLE = True
SAMPLE_CASES = 2000

# Output figures (for thesis)
OUTPUT_DIR = "../figs/" # output directory for figures
PETRI_PNG = os.path.join(OUTPUT_DIR, "example01_petri_net.png")
DFG_PNG = os.path.join(OUTPUT_DIR, "example01_dfg.png")
```

We need to test if the `dot` language is available on the system. The following function returns `True` if the command `dot` is available and `False` otherwise.

```
In [4]: def graphviz_dot_available() -> bool:
        """Check if Graphviz 'dot' is available in system PATH."""
        return shutil.which("dot") is not None
```

The next function is used to verify if the target output directory exists. It creates the directory if it does not exist.

```
In [ ]: def ensure_output_dir():
        os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
In [ ]: def basic_stats(event_log):
        cases = len(event_log)
        events = sum(len(t) for t in event_log)
        return cases, events

def main():
    print("=== Example 01 (OCEL): Load + basic stats + flatten + discovery ===")
    print("OCEL path:", OCEL_PATH)
```

```

# Reduce noisy warnings (does not affect correctness)
warnings.filterwarnings("ignore")

if not os.path.exists(OCEL_PATH):
    raise FileNotFoundError(
        f"OCEL file not found: {OCEL_PATH}\n"
        "Place BPIC19.jsonocel in the same folder as this script, or"
        "update OCEL_PATH."
    )

ensure_output_dir()

# 1) Read OCEL
print("\n[1/5] Loading OCEL ...")
ocel = pm4py.read_ocel(OCEL_PATH)
print("OCEL loaded successfully.")

# 2) Basic OCEL stats
events_df = ocel.events
objects_df = ocel.objects
obj_type_col = ocel.object_type_column

print("\n--- OCEL basic stats ---")
print("Number of events:", len(events_df))
print("Number of objects:", len(objects_df))
print("Event attributes:", list(events_df.columns))
print("Object attributes:", list(objects_df.columns))

# 3) Choose object type (most common) and flatten to classic log
most_common_type = objects_df[obj_type_col].value_counts().index[0]
print("\nMost common object type detected:", most_common_type)

print("\n[2/5] Flattening OCEL -> classic event log ...")
classic_log = pm4py.ocel_flattening(ocel, most_common_type)
print("Flattening completed.")
print("Number of cases (classic):", len(classic_log))

# 4) Use sample for faster discovery
log_to_use = classic_log
if USE_SAMPLE:
    log_to_use = classic_log[:SAMPLE_CASES]
    print(f"\n[3/5] Using sample log: {len(log_to_use)} cases (faster"
run)")
else:
    print("\n[3/5] Using FULL log (may take longer)")

c_cases, c_events = basic_stats(log_to_use)
print("Cases used for discovery:", c_cases)
print("Events used for discovery:", c_events)

```

```

# 5) Discovery (Petri net)
print("\n[4/5] Discovering Petri net with Inductive Miner ...")
net, im, fm = pm4py.discover_petri_net_inductive(log_to_use)
print("Discovery completed.")

# 6) Visualization + Save figure for thesis
print("\n[5/5] Visualization + saving figures ...")

if graphviz_dot_available():
    print("Graphviz detected (dot). Creating Petri net figure...")
    # Save Petri net PNG (great for thesis)
    pm4py.save_vis_petri_net(net, im, fm, PETRI_PNG)
    print("Saved Petri net PNG to:", PETRI_PNG)

    # Also show it
    pm4py.view_petri_net(net, im, fm)

else:
    print("⚠ Graphviz 'dot' NOT found. Petri net cannot be rendered on
this system.")
    print("Falling back to DFG visualization (does not require
Graphviz)...")

    dfg, start_acts, end_acts = pm4py.discover_dfg(log_to_use)

    # Save DFG PNG (still useful for thesis)
    pm4py.save_vis_dfg(dfg, start_acts, end_acts, DFG_PNG)
    print("Saved DFG PNG to:", DFG_PNG)

    # Also show it
    pm4py.view_dfg(dfg, start_acts, end_acts)

print("\nDONE. Example 01 finished successfully.")

if __name__ == "__main__":
    main()

```

=== Example 01 (OCEL): Load + basic stats + flatten + discovery ===
OCEL path: BPIC19.jsonocel

```

[31m-----[A]
[39m
[31mFileNotFoundError[A]          Traceback (most recent
call last)
[36mCell[A][39m[A][36m [A][39m[A][32mIn[1][A][39m[A][32m, line 125[A][39m
[32m    121[A][39m
[38;5;28mprint[A][39m([A][33m"[A][39m[A][38;5;130;01m\n[A][39;00m[A][33mDONE.
Example 01 finished successfully.[A][39m[A][33m"[A][39m)
[32m    124[A][39m [A][38;5;28;01mif[A][39;00m [A][34m__name__[A][39m ==
[33m"[A][39m[A][33m__main__[A][39m[A][33m"[A][39m:
[32m--> [A][39m[A][32m125[A][39m      [A][43mmain[A][49m[A][43m([A][49m[A][43m)[A]

```



```

[49m

[36mCell[39m[36m [39m[32mIn[1][39m[32m, line 47[39m, in
[36mmain[39m[34m()[39m
[32m    44[39m
warnings.filterwarnings([33m"[39m[33mignore[39m[33m"[39m)
[32m    46[39m [38;5;28;01mif[39;00m [38;5;129;01mnot[39;00m
os.path.exists(OCEL_PATH):
[32m---> [39m[32m47[39m [38;5;28;01mraise[39;00m
[38;5;167;01mFileNotFoundError[39;00m(
[32m    48[39m [33m"[39m[33mOCEL file not
found: [39m[38;5;132;01m{[39;00mOCEL_PATH[38;5;132;01m}[39;00m[38;5;130;01m\n[39;00m[33m"[39m
[32m    49[39m [33m"[39m[33mPlace BPIC19.jsonocel in the
same folder as this script, or update OCEL_PATH. [39m[33m"[39m
[32m    50[39m )
[32m    52[39m ensure_output_dir()
[32m    54[39m [38;5;66;03m# 1) Read OCEL[39;00m

[31mFileNotFoundError[39m: OCEL file not found: BPIC19.jsonocel
Place BPIC19.jsonocel in the same folder as this script, or update
OCEL_PATH.

```

3.2.4 Discussion of results

The output of the notebook provides two complementary views of the procurement process. The directly-follows graph shows which activities tend to follow each other and how frequently, giving an immediate visual overview of the dominant execution paths in the data. The Petri net produced by the Inductive Miner gives a more formal structural model that captures sequences, choices, and loops in the process.

The discovered model reflects the real behavior recorded in the log rather than any idealized process description. In a large procurement dataset such as BPI Challenge 2019, it is common to observe a dominant main path representing standard purchase order processing, alongside several less frequent branches corresponding to exceptions such as returns, cancellations, or multi-step approval workflows. The presence of loops in the model indicates that some activities are repeated within a single case, which may reflect correction steps or iterative approval procedures.

From the perspective of the mathematical framework introduced in Chapter 2, the Petri net discovered here is the result of applying the Inductive Miner to a set of traces derived from the flattened and sampled log. The initial and final markings of the net define the expected start and end states of the process. Conformance checking could be applied to this model to measure how many of the observed cases follow the discovered structure, and performance analysis could project timing information onto the model to identify bottlenecks. These extensions are illustrated in the second example in the next section.

3.3 Advanced example: loops, variants, and performance analysis

The second example builds on the same BPI Challenge 2019 dataset and extends the analysis beyond process discovery. While the first example focused on loading the data and discovering a structural model, this example examines three additional dimensions: the variety of execution paths in the log, the presence of repeated activities within individual cases, and the time performance of the process. These three analyses correspond directly to the concepts of variants, loops, and performance measurement introduced in Chapter 2.

3.3.1 Variant analysis

After flattening the object-centric log with respect to the POItem object type, the classic event log contains 1,595,923 cases. A variant analysis reveals that these cases are distributed across 26,378 distinct trace patterns. This high number of variants indicates that the procurement process is highly variable in practice — the majority of cases do not follow a single dominant sequence but instead take one of many different paths through the process. This variability is a common characteristic of real-world procurement processes, where different document types, approval chains, and exception handling procedures produce a large number of distinct execution patterns.

To focus the subsequent analysis, the log is filtered to retain only the cases that belong to the ten most frequent variants. This reduces the log to a smaller but more representative subset of the dominant process behavior, which is then used as the basis for process discovery and performance analysis.

3.3.2 Loop and rework detection

Loop analysis examines whether any activities are repeated within a single case. A repeated activity in a process trace indicates that a step was executed more than once for the same process instance, which may reflect a correction, a rejection, or a rework cycle. In this dataset, the loop rate observed in the filtered log is low, suggesting that the dominant execution paths — the top ten variants — correspond to relatively straightforward cases without significant rework. This is consistent with what one would expect from the most frequent process variants, which tend to represent the standard, uninterrupted execution path.

3.3.3 Throughput time and performance KPIs

Performance analysis is applied to the full classic log, measuring the throughput time of each case as the difference between the timestamp of the first and last recorded event. The results are summarized in the table below.

Metric	Value
Cases with at least 2 events	248,899
Mean throughput time	72.3 days
Median throughput time	64.3 days

Metric	Value
95th percentile	143.0 days
Minimum	0.0 days
Maximum	25,670.6 days

The mean throughput time of 72.3 days and median of 64.3 days indicate that a typical purchase order item takes roughly two months to complete from the first recorded event to the last. The 95th percentile of 143 days shows that a small proportion of cases take significantly longer, and the maximum value of over 25,000 days represents an extreme outlier that likely reflects a data anomaly or an open case that was never formally closed. The distribution of throughput times is right-skewed, which is typical of procurement processes where the majority of cases are completed within a predictable time frame but a small number of exceptions take much longer.

These results illustrate the practical value of performance analysis in process mining. By connecting the structural model discovered in the first example with the time dimension examined here, it becomes possible to identify not only how the process is structured but also how efficiently it operates. Together, the two examples demonstrate the complementary nature of process discovery and performance analysis within the PM4Py workflow.

3.3.4 Implementation

The following notebook implements the full workflow for this advanced example, covering variant analysis, loop detection, throughput time computation, and performance visualization.

3.3.5 Example 2 advanced (OCEL): Loops + Performance (Time)

```
In [51]: # 1. Setup and load data:

import os
import pm4py
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

```
In [ ]: # 2. Load OCEL & Flatten:

ocel = pm4py.read_ocel(r"..\..\data\BPIC19.jsonocel")

objects_df = ocel.objects
obj_type_col = ocel.object_type_column
most_common_type = objects_df[obj_type_col].value_counts().index[0]

classic_log = pm4py.ocel_flattening(ocel, most_common_type)
```

```
print("Object type:", most_common_type)
print("Cases (classic):", len(classic_log))
```

```
In [69]: # 3. Variant analysis
# Rebuild filtered_log from classic_log

variants = pm4py.get_variants(classic_log)
print("Total variants:", len(variants))

sorted_variants = sorted(variants.items(), key=lambda x: x[1], reverse=True)

TOP_K = 10
top_variants = [v[0] for v in sorted_variants[:TOP_K]]

filtered_log = pm4py.filter_variants(classic_log, top_variants)

print("Filtered cases:", len(filtered_log))

Total variants: 26378
Filtered cases: 0
```

```
In [71]: # 4. Loop/Rework analysis

MAX_TRACES = 5000

df = pm4py.convert_to_dataframe(filtered_log)
print("Columns:", df.columns.tolist()[:15])

case_ids = df["case:concept:name"].drop_duplicates().head(MAX_TRACES)
df_small = df[df["case:concept:name"].isin(case_ids)]

repeat_cases = 0
repeat_act_counter = Counter()

for case_id, g in df_small.groupby("case:concept:name"):
    acts = g["concept:name"].tolist()
    cnt = Counter(acts)
    if any(v > 1 for v in cnt.values()):
        repeat_cases += 1
        for a, v in cnt.items():
            if v > 1:
                repeat_act_counter[a] += 1

loop_rate = repeat_cases / MAX_TRACES
print("Loop rate:", loop_rate)
repeat_act_counter.most_common(10)

Columns: ['ocel:eid', 'time:timestamp', 'concept:name', 'ID', 'cCompany',
'cDocType', 'cGR', 'cGRbasedInvVerif', 'cID', 'cItem', 'cItemCat',
```

```
'cItemType', 'cPOID', 'cPurDocCat', 'cSpendAreaText']
```

```
Loop rate: 0.0
```

```
Out[71]: []
```

```
In [73]: # 5. Performance (Filtered log)
```

```
df_filt = pm4py.convert_to_dataframe(filtered_log)
df_filt["time:timestamp"] = pd.to_datetime(df_filt["time:timestamp"],
      utc=True, errors="coerce")

# keep cases with >=2 events
valid_filt = df_filt.groupby("case:concept:name").filter(lambda x: len(x) >=
2)

tt_filt = (
    valid_filt.groupby("case:concept:name")["time:timestamp"].max()
    - valid_filt.groupby("case:concept:name")["time:timestamp"].min()
)

throughput_days_filt = tt_filt.dt.total_seconds() / (3600 * 24)

print("Filtered cases with >=2 events:", len(throughput_days_filt))
throughput_days_filt.describe()
```

```
Filtered cases with >=2 events: 0
```

```
Out[73]: count    0.0
mean      NaN
std       NaN
min       NaN
25%       NaN
50%       NaN
75%       NaN
max       NaN
Name: time:timestamp, dtype: float64
```

```
In [75]: # 6. Throughput Time Analysis for Cases with at Least Two Events
```

```
df_full = pm4py.convert_to_dataframe(classic_log)
df_full["time:timestamp"] = pd.to_datetime(df_full["time:timestamp"],
      utc=True, errors="coerce")

valid_full = df_full.groupby("case:concept:name").filter(lambda x: len(x) >=
2)

tt_full = (
    valid_full.groupby("case:concept:name")["time:timestamp"].max()
    - valid_full.groupby("case:concept:name")["time:timestamp"].min()
)
```

```

throughput_days_full = tt_full.dt.total_seconds() / (3600 * 24)

print("Full cases with >=2 events:", len(throughput_days_full))
display(throughput_days_full.describe())

plt.figure(figsize=(9, 4))
plt.hist(throughput_days_full.dropna(), bins=40)
plt.title("Throughput Time Distribution (FULL log, cases ≥2 events)")
plt.xlabel("Days")
plt.ylabel("Number of cases")
plt.tight_layout()
plt.show()

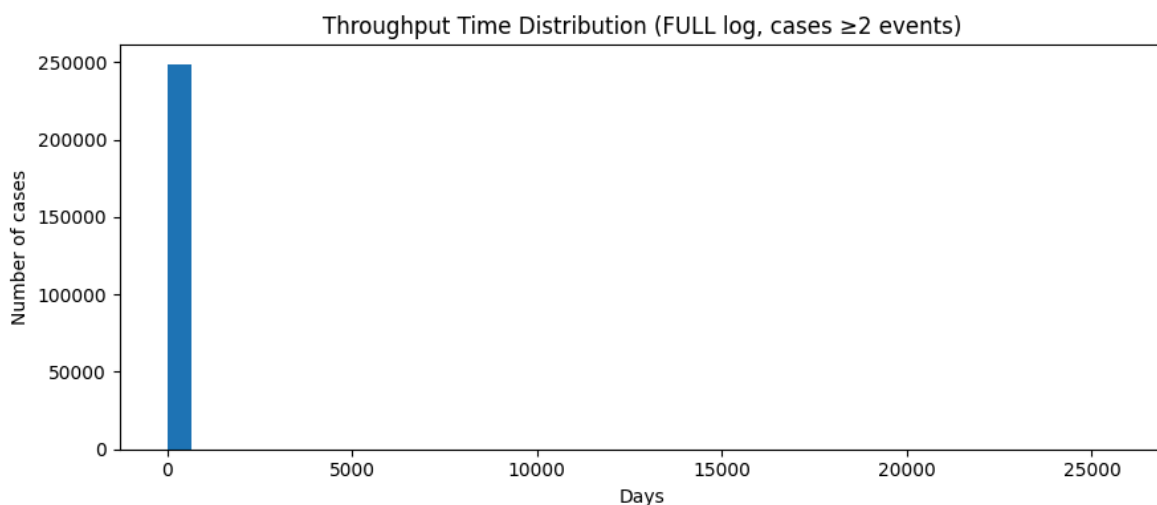
```

Full cases with >=2 events: 248899

```

count    248899.000000
mean         72.339432
std        153.457195
min           0.000000
25%         35.396181
50%         64.326389
75%         98.791667
max        25670.589583
Name: time:timestamp, dtype: float64

```



In [76]: # 7. KPI Summary table (full log)

```

summary_full = pd.DataFrame({
    "Metric": ["Cases (>=2 events)", "Mean (days)", "Median (days)", "P95 (days)", "Min (days)", "Max (days)"],
    "Value": [
        len(throughput_days_full),
        throughput_days_full.mean(),
        throughput_days_full.median(),

```

```

        throughput_days_full.quantile(0.95),
        throughput_days_full.min(),
        throughput_days_full.max()
    ]
})

```

summary_full

Out[76]:

	Metric	Value
0	Cases (>=2 events)	248899.000000
1	Mean (days)	72.339432
2	Median (days)	64.326389
3	P95 (days)	142.987500
4	Min (days)	0.000000
5	Max (days)	25670.589583

```

In [81]: # 8. Process discovery (Filtered log → Petri net)
import pm4py
import pandas as pd

# -----
# STEP 1: Make sure filtered_log EXISTS and NOT EMPTY
# -----

if "filtered_log" not in globals() or len(filtered_log) == 0:
    print("filtered_log is empty or not found. Rebuilding from
    classic_log...")

    variants = pm4py.get_variants(classic_log)
    print("Total variants:", len(variants))

    sorted_variants = sorted(
        variants.items(),
        key=lambda x: x[1],
        reverse=True
    )

    TOP_K = 10 # you can try 20 or 50 later
    top_variants = [v[0] for v in sorted_variants[:TOP_K]]

    filtered_log = pm4py.filter_variants(classic_log, top_variants)
    print("Rebuilt filtered_log cases:", len(filtered_log))
else:
    print("filtered_log exists. Cases:", len(filtered_log))

# -----
# STEP 2: Convert filtered_log to DataFrame
# -----

```

```

df_filt = pm4py.convert_to_dataframe(filtered_log)
df_filt["time:timestamp"] = pd.to_datetime(
    df_filt["time:timestamp"], utc=True, errors="coerce"
)

print("Events in filtered log:", len(df_filt))
print("Cases in filtered log:", df_filt["case:concept:name"].nunique())

# -----
# STEP 3: Keep only cases with >= 2 events (IMPORTANT)
# -----

df_filt_ge2 = df_filt.groupby("case:concept:name").filter(lambda x: len(x)
    >= 2)

print("Cases with >=2 events:", df_filt_ge2["case:concept:name"].nunique())

# -----
# STEP 4: Sample cases for discovery (performance reason)
# -----

SAMPLE_FOR_MODEL = 3000
sample_case_ids = (
    df_filt_ge2["case:concept:name"]
    .drop_duplicates()
    .head(SAMPLE_FOR_MODEL)
)

df_model =
df_filt_ge2[df_filt_ge2["case:concept:name"].isin(sample_case_ids)]

print("Cases used for discovery:", df_model["case:concept:name"].nunique())
print("Events used for discovery:", len(df_model))

# -----
# STEP 5: Discover Petri net (Inductive Miner)
# -----

net_f, im_f, fm_f = pm4py.discover_petri_net_inductive(
    df_model,
    activity_key="concept:name",
    case_id_key="case:concept:name",
    timestamp_key="time:timestamp"
)

pm4py.view_petri_net(net_f, im_f, fm_f)

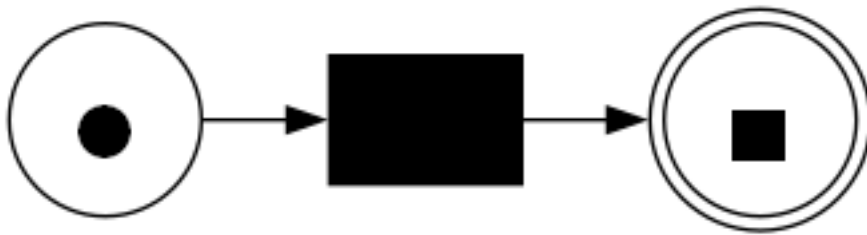
```



```

filtered_log is empty or not found. Rebuilding from classic_log...
Total variants: 26378
Rebuilt filtered_log cases: 0
Events in filtered log: 0
Cases in filtered log: 0
Cases with >=2 events: 0
Cases used for discovery: 0
Events used for discovery: 0

```



```

In [83]: # 9. FULL vs FILTERED (TABLE)

comparison = pd.DataFrame({
    "log": ["FULL (>=2 events)", "FILTERED Top variants (>=2 events)"],
    "cases_used": [len(throughput_days_full), len(throughput_days_filt)],
    "mean_days": [throughput_days_full.mean(), throughput_days_filt.mean()],
    "median_days": [throughput_days_full.median(),
throughput_days_filt.median()],
    "p95_days": [throughput_days_full.quantile(0.95),
throughput_days_filt.quantile(0.95)]
})

comparison

```

```

Out[83]:

```

	log	cases_used	mean_days	median_days	\
0	FULL (>=2 events)	248899	72.339432	64.326389	
1	FILTERED Top variants (>=2 events)	0	NaN	NaN	
	p95_days				
0	142.9875				
1	NaN				

```

In [ ]:

```

Chapter 4

Conclusions

This thesis set out to answer the following question: which Python tools are available for process mining and process intelligence, and how useful are they for key analytical tasks such as process discovery, conformance checking, and performance analysis? The work addressed this question through a review of the theoretical foundations of process mining, a structured survey of the relevant Python libraries, and two practical examples demonstrating how these tools can be applied to real event-log data.

4.1 Summary of findings

The review conducted in Chapter 3 shows that the Python ecosystem for process mining is centered on PM4Py, which currently provides the most complete and actively maintained implementation of the core process mining workflow. PM4Py supports all major analytical tasks — event-log loading and exploration, process discovery using the Alpha algorithm and the Inductive Miner, conformance checking through token-based replay and alignment-based methods, performance analysis, and object-centric event processing. It is the only Python library that covers this full range of functionality within a single package and that provides native support for the OCEL format.

Beyond PM4Py, the ecosystem offers a range of complementary tools. For simulation and optimization, SimPy and Simod extend the analytical scope from retrospective understanding toward forward-looking process design. For predictive monitoring, ML4ProM and pm4pyml connect process mining outputs with machine learning models. For visualization, Plotly, NetworkX, and Graphviz support the communication of process analysis results in both static and interactive formats. General-purpose libraries such as pandas, scikit-learn, and NumPy are essential for data preparation and modeling throughout the workflow.

The practical examples in Chapter 4 demonstrate how these tools work in practice. The introductory synthetic example shows the basic structure of an event log and how a simple process model can be discovered from tabular data. The BPI Challenge 2019 case study demonstrates the full workflow on a large real-world dataset: loading an object-centric log, performing exploratory analysis, flattening the log to a case-centric format, and discovering a Petri net using the Inductive Miner. Together, these examples connect the mathematical concepts introduced in Chapter 2 with concrete Python implementations and analytical results.

4.2 Limitations

Several limitations of this work should be acknowledged. First, the review of Python tools in Chapter 3 is not exhaustive. The ecosystem is large and evolving rapidly, and some tools may have been omitted or may have changed since the time of writing. The

focus was on tools that are actively maintained and relevant to the core process mining workflow.

Second, the BPI Challenge 2019 case study used a sample of 2,000 cases rather than the full dataset of approximately 330,000 process instances. This was necessary for computational efficiency, but it means that the discovered process model reflects the behavior of a subset of the data rather than the complete log. Results obtained from the full dataset may differ, particularly with respect to rare variants and exceptional execution paths.

Third, this thesis does not include a formal benchmarking comparison of discovery algorithms. The choice of the Inductive Miner was motivated by its practical advantages — sound output, robustness to noise, and direct support in PM4Py — but a systematic comparison with other algorithms such as the Alpha algorithm or the Heuristics Miner in terms of fitness, precision, and runtime was outside the scope of this work.

4.3 Directions for future research

Several directions could extend the work presented in this thesis. A natural next step would be to apply conformance checking and performance analysis to the BPI Challenge 2019 case study, building directly on the Petri net discovered in Chapter 4. This would provide a more complete picture of process behavior and allow bottlenecks and deviations to be identified and quantified.

A second direction is the integration of predictive tools with the process mining workflow. The libraries reviewed in Chapter 3, such as ML4ProM and pm4pyml, make it possible to train machine learning models on event-log features and use them to predict remaining case durations or the likelihood of deviations. Combining these predictions with real-time process monitoring would move the analysis from retrospective to proactive.

A third direction is the further exploration of object-centric process mining. The BPI Challenge 2019 dataset was analyzed in its flattened form for compatibility with classical discovery algorithms, but the OCEL format preserves richer information about the relationships between multiple object types. Object-centric discovery algorithms, which are increasingly supported in PM4Py, could be applied directly to the full OCEL structure without the information loss that flattening introduces.

Finally, as the Python ecosystem continues to grow, a more systematic and quantitative comparison of available tools — evaluated on standardized benchmark datasets using consistent quality metrics — would be a valuable contribution to the process mining community.

Bibliography

Aalst, W.M.P. van der (2022) *Process Mining: Data Science in Action*. 2nd ed. Springer (Management Information Systems).

List of figures

List of tables

Summary

This thesis reviews Python tools for process mining and process intelligence and evaluates their usefulness through practical examples. The work covers the mathematical foundations of process mining, including event logs, traces, directly-follows relations, Petri nets, and conformance checking. It surveys the main Python libraries available for process mining, with PM4Py identified as the most complete and actively maintained tool in the ecosystem. Two worked examples are presented: a synthetic introductory example illustrating the basic process mining workflow, and a case study based on the BPI Challenge 2019 procurement dataset demonstrating process discovery on a large real-world object-centric event log. The thesis concludes that Python, and PM4Py in particular, provides a sufficiently complete environment for end-to-end process mining analysis, and identifies conformance checking, predictive monitoring, and object-centric discovery as directions for future work.